

电机驱动板系统架构

foc_mower 程序框架说明（架构 + 数据流）

LDROBOT 割草机电机 FOC 驱动固件（GD32F303，净室重建，定点为主）。本文讲清分层架构、依赖规则、指令怎么一步步变成 PWM、以及关键数据结构归属。
配套：[开发与调试指南](#) · [更新记录](#) · [接续上下文](#)

1. 分层架构

依赖只向下/向内、无环。下面两张图配合看：**§1.1 分层总览**——每层有什么(含横切件清单)；
§1.2 调用/依赖关系——谁调用谁。切记 `middleware/lib` 不是"夹在 application 与 bsp 之间的转发层"，
而是 application 旁挂、只做纯计算的算法库(§1.3 详述)。sdk 为整车维护的跨板通信协议库（原样引入）。

1.1 分层总览（层 × 模块 × 横切件）

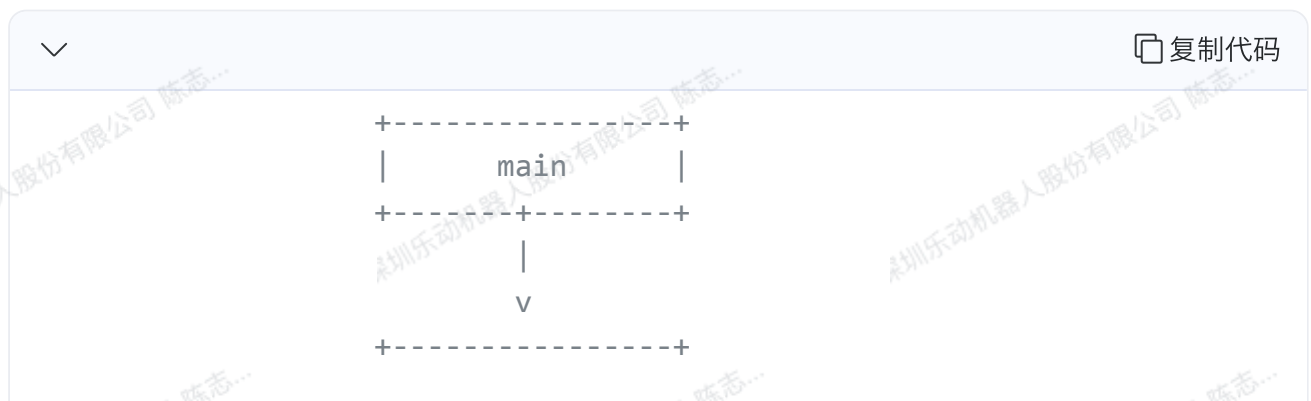
复制代码

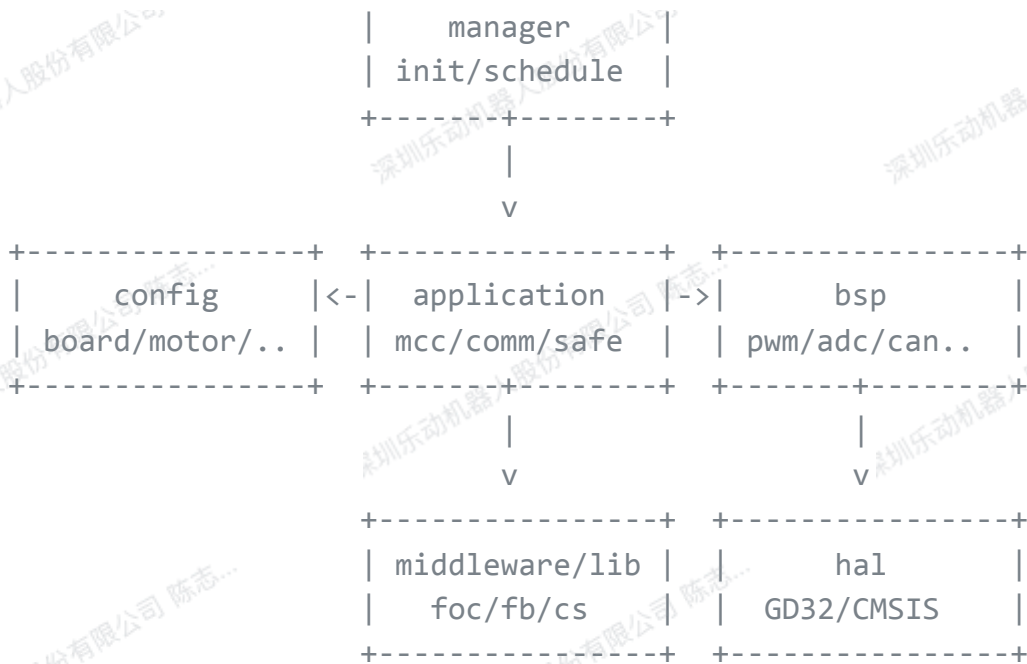
host / 算法板 / 主板 (CAN bus) --cmd-->			
+-----+-----+-----+-----+			
-----+-----+-----+-----+			
layer	modules		
common	os	config	
+-----+-----+-----+-----+			
-----+-----+-----+-----+			
manager	app_main : init + schedule periodic jobs		
crc	os_port	app	
+-----+-----+-----+-----+			
ringbuf	sched	board	
application	motor_app : mcc/current_isr/speed/fsm/protect		
filter	bare/	motor	



本表看每层有什么；实际"谁调用谁"见 §1.2 框图——注意 `middleware/lib` 并非 `application` `n→bsp` 的中间转发层，而是 `application` 旁挂的算法库（本表把它排在 `application` 下方只是按"抽象层级"列，非调用顺序）。

1.2 调用 / 依赖关系（框图：箭头 = 依赖/调用方向）





横切(不在上面主链、被多处调用):

common : crc / filter / ramp / ringbuf / range_check 被 application
与 middleware/lib 共用
os : sched / os_port 被 manager /
application 调用; 实时电流环不经它
sdk : 整车 CAN 协议结构/常量 主要被

► **application 是组合根**: 一手向下调 middleware/lib 做纯计算、一手调 bsp 读写外设, 并把 bsp 读写

以函数指针(cs_io_t / fb_io_t)注入给 lib——故 lib 零 bsp 依赖、bsp 与 lib 互不直接依赖。

► config 被各层读取 (含 lib, 经 foc_config.h 取 app_cfg.h), 图中只画到 application 以免连线过密。

► 精确依赖边: manager→{application, os, bsp(仅初始化), config}、application→{lib, bsp, sdk, common, os, config}、
middleware/lib→{common, config}、middleware/user→{bsp, config}、bsp→{hal, config} ; 无下层反向依赖。

1.3 common 与 middleware/lib 的区别 (回答"lib 能并入 common 吗"——不能)

两者都"被上层调、自身不碰硬件", 容易混为一谈, 但层级与用户面不同:



复制代码

依赖次序: application → middleware/lib → common → config
 └──────────────────────────────────┘
 common (也直接用)

middleware/lib : 电机领域算法库(FOC/估算/电流重构)。消费者=仅 application; 自身还要用 common。

→ 位于 common 之上、领域专用、单一消费者。

common : 通用基础工具(crc/filter/ramp/ringbuf/range_check)。消费者=application + middleware/lib。

→ 位于 lib 之下、跨域通用、多消费者。

common 更底层、更通用、被多层复用；lib 更上层、只服务电机域、只被 application 调用。故不同层——

若把 lib 塞进 common, 会让"通用工具"沾上电机域依赖 (且 lib→common 的依赖方向会反过来), 破坏 common

的通用性与可复用性。所以 lib 保持在 middleware（算法库），不并入 common。

1.4 各层职责（键为层名，冒号后含职责 / 禁止 / 是否 MCU 相关）

 复制代码

manager : 应用入口、周期作业注册与调度; 不碰寄存器/不写环路; 非 MCU 相关

application : 电机门面(mcc)/通信/安全/参数, 组合根(在此把 bsp 注入 lib); 不直接碰寄存器; 非 MCU 相关

middleware.lib : 纯软件算法/协议库(FOC/估算/电流重构), 可单测/可打 .lib;
仅 application 调用、不依赖 bsp/hal(改注入); 非 MCU

middleware.user : 移植适配层, 为纯 lib 绑定 bsp(如温度换算读 ADC); 不承载业务/算法; 经 bsp 触硬件

bsp : 板级配置 + 外设访问封装(按外设分组); 不承载应用/算法; ★MCU 相关唯一层

hal : 芯片厂商标准外设库/CMSIS/启动文件；保持原版不改；MCU 相关

sdk : 整车跨板 CAN 协议库(单一真源, 随上游同步); 不改动内容
--- 横切件(不反向依赖上层、不侵入实时电流环) ---

common : 通用工具(crc/filter/ramp/ringbuf/range_check); 被 application 与 middleware/lib 调用

os : OS 抽象(裸机·FreeRTOS 二选一); 被 manager/application 调用; 实时电流环不经它

config : 分层配置(纯宏无代码); 被所有层读取(含 lib—经
foc_config.h 取 app_cfg.h)

依赖铁律（已校验）

1. 依赖只向下/向内、无环：到硬件的主脊柱是 `main → manager → application → bsp → hal`；`middleware/lib` 不在脊柱上，是 `application` 调用的纯算法库(不碰 `bsp`)；横切件依赖次序 `application → middleware/lib → common → config`（`application` 也直接用 `common/config`）。无下层反向 `include` 上层。
2. 同层向上用回调：如 CAN 收帧经 `bsp_can_set_rx_callback` 回调 `commapp`、ADC EOC 经 `bspadcseteoc_callback` 回调 `mcc`。
3. `middleware/lib` 纯软件：不直接依赖 `bsp/hal`。需要读硬件的（电流原始采样、霍尔电平、编码器计数）由 `application(mcc)` 以函数指针注入（`cs_io_t` / `fb_io_t`），`lib` 只调注入的接口。这样 `lib` 可移植、可单测、可打成 `.lib`。
4. 实时电流环不经 OS：`mcc_current_isr` 只用 `application/middleware/bsp`，裁掉 FreeRTOS 不影响电机控制。
5. 换芯片只改 `bsp/` + 换 `hal/`；换电机只改 `config/motor_cfg.h`。

2. 目录结构



复制代码

```
foc_mower/
├── main.c                独立板入口: system_clock_init → app_init →
app_run
├── manager/              app_main: 初始化各子系统 + 注册周期作业(ctrl
1ms/comm 1ms/drive.safety 10ms/wdg 50ms)
├── application/
│   ├── motor_app/        mcc(门面/电流环ISR/速度环/状态机/保护) +
motor_obj(motor_t/motor_data_t)
│   │                   + motion_ctrl(差速) + drive_ctrl +
│   │                   mcc_ident(参数自辨识 R/L/霍尔偏角, FUNC_AUTO_ID)
│   ├── comm_app/         comm_svc(SDK CAN 协议适配器) + comm_api.h
│   ├── safety_app/       safety(通信丢失看门狗 → mcc_stop_all)
│   └── param_app/        param(Flash 参数块 magic/版本/CRC; 来源
PARAM_SOURCE 选 .h/Flash)
├── middleware/
│   ├── lib/              纯软件库(依赖注入 bsp):
│   └── foc/
foc_types/foc_config/sine_lut/clarke_park/pi_ctrl/field_weaken/foc_core(
电流环引擎)
```

```

|   |   | — estimator/      feedback(工厂) +
fb_hall/fb_encoder/fb_sensorless + smo/pll
|   |   | — curr_sense/      相电流采样与重构(多种方式)
|   |   | — user/sensor/      ntc_table(温度换算, 绑定 bsp_adc)
| — bsp/                      硬件驱动(GD32F303): core(时钟120M/SysTick) ·
pwm(含SVPWM/死区) · adc · encoder ·
|                               can · uart · gpio · timer · flash · gate ·
wdg
| — hal/                      GD32F30x 标准外设库 / CMSIS / startup (原版不改)
| — sdk/                      整车 CAN 协议库:
uart_comm_protocol/robot_comm_can_protocol.h + data_*.h
| — common/                   crc · ringbuf · filter · ramp
| — config/                   app · board · motor · motion · protect ·
comm · func + app_version
| — os/                       os_port · sched + bare/ +
freertos/(FreeRTOSConfig + kernel V11.3.0)
| — doc/ · test/              文档 · 测试(正式工程不含)

```

3. 数据流：指令 → PWM（以速度环为例）

系统是双速率：外环(速度/位置) 1kHz 生成电流给定，内环(电流) 在 PWM 频率(10kHz)的 ADC 中断里执行。

3.1 指令进入（事件驱动，收到 CAN 帧时）

▼ 📄 复制代码

```

CAN 收帧(ISR) → 入环形缓冲
[bsp/can/can.c → comm_svc.c can_rx_cb]
→ comm_poll()(1ms 任务上下文)出队 → CanProto_DecodeId
[application/comm_app/comm_svc.c]
→ 按 mode_id 分发:
    MODE_ID_CTRL_REAR(0x022) {左轮速, 右轮速}(rpm) → set_wheel →
mcc_set_mode(SPEED)+set_speed_ref+enable
    MODE_ID_ESTOP/BRAKE → mcc_stop_all()
    MODE_ID_QUERY_* → 置应答标志(快/中/慢包)

```

`mcc_set_*` 只写 `s_motors[id]` 字段，不做实时动作——指令与执行解耦。

3.2 外环：速度环 + 状态机 (1kHz `mcc_tick_1k`)

[复制代码](#)

- ① 采慢变量: `bus_mv`、温度、`speed_rpm`(由 `fb->get_speed` 换算平滑)、`position_cnt`
- ② `check_faults()`: 过流(去抖)/过压/欠压/过温/堵转/霍尔 → `fault_flags`
- ③ `fsm_step()`: `INIT`→`IDLE`→(`ALIGN` 无感开环拖)→`RUN`→`FAULT`
 `RUN + SPEED`: `foc.iq_ref = pi_run(pi_speed, ramp(speed_ref) - speed_rpm)`
- ④ 温度折返限流 `current_limit()` 钳 `iq` 上限; 更新 `status_word`

速度环唯一产出 = `m->foc.iq_ref` (`q` 轴电流给定)。

3.3 内环：电流环 (10kHz, ADC 注入中断 `mcc_current_isr`)

[复制代码](#)

ADC 注入转换完成中断 → EOC 回调 → `mcc_current_isr(id)`
[`bsp_adc/adc.c` → `mcc.c`]

- ① `i = cs->read()` 三相电流(mA)
[`curr_sense`, 经注入的 `bsp_adc` 读原始值]
- ② `clarke` → `fb->update()` 更新反馈观测
- ③ `angle = fb->get_angle()` 叠加 `hall_angle_offset/speed_dir`
- ④ `v_ab = foc_core_run(foc, angle, i, espeed)`
[`foc_core`: `Clarke/Park/电流PI/弱磁/圆限幅/逆Park`, 输出归一化电压]
- ⑤ `bsp_pwm_set_voltage(id, v_ab.α, v_ab.β, i.a,i.b,i.c)` [pwm: 逆
`Clarke+零序注入SVPWM+死区补偿`→写比较寄存器]

电压全程归一化 (`FOC_V_FULLSCALE=32767` =满调制), 母线电压不进入 `FOC/SVPWM` 运算 (与电压平台无关), 母线波动由电流环吸收。四种模式 (电压开环/电流/速度/位置) 最终都收敛到同一电流环内核 `foc_core_run`, 仅"给定从哪来"不同。

4. 关键数据结构归属

键为结构名(冒号后=所在文件), 下一行缩进为角色:

[复制代码](#)

`motor_t` : `application/motor_app/motor_obj.h`

一路电机全部：参数 p / FOC 引擎 foc / 速度·位置 PI / 反馈·采样·预驱句柄 / 指令 / data / 状态机

motor_data_t : application/motor_app/motor_obj.h

运行数据聚合(角度/电流/电压/转速/位置/母线/温度/状态字/故障)；快变量电流环更新、慢变量 1kHz 更新；对外单一数据源

foc_core_t : middleware/lib/foc/foc_core.h

电流环引擎(模式 / PI / 给定 / dq / $\alpha\beta$ / 弱磁)

feedback_t + fb_io_t : middleware/lib/estimator/feedback.h

位置反馈统一接口(工厂 fb_create) + 注入的传感器读取钩子

curr_sense_t + cs_io_t : middleware/lib/curr_sense/curr_sense.h

电流采样接口(工厂 cs_create) + 注入的 ADC 原始采样钩子

gate_driver_t : bsp/gate/gate_driver.h

栅极预驱接口(工厂)

可插拔约定：feedback_t / curr_sense_t / gate_driver_t 均为"函数指针结构体 + xxx_create() 工厂"，编译期 FEAT_* 裁剪、运行期选择。lib 需要的 bsp 读取 (bsp_adc_get_phase_raw / bsp_enc_hall_state / bsp_enc_get_count 等) 由 mcc 在 cs_create / fb_create 时以 cs_io_t / fb_io_t 注入，lib 本身零 bsp 依赖。

5. 单位约定（全工程一致）

- 电角度：uint16 整周 65536；正余弦 Q15。
- 电流：mA (Clarke/Park 用 Q15_SMUL 降标度 FOC_FIX_SHIFT=3)，int32 量程约 $\pm 384A$ 。
- 电压：**FOC 内部归一化** (FOC_V_FULLSCALE=32767 对应满调制 $V_{dc}/\sqrt{3}$ ，FOC_V_BUS_NORM=56755)；对外遥测/电压给定亦为归一化 ± 32767 (统一标度，非 mV)。
- 转速：机械 rpm。温度：0.1°C。PI 增益：Q(PI_GAIN_SHIFT=12) 定点。

6. 单/双电机与 OS 可裁剪

- config/app_cfg.h 的 MOTOR_COUNT (1~2)：静态分配 s_motors[]，中速环 for 各轴、快环各轴独立 ADC 中断。
- USE_RTOS=0 裸机超循环 / =1 FreeRTOS(V11.3.0)，业务不变；实时快环在中断中不依赖 OS。
- 组件按 config 的 FEAT_* 编译期裁剪；未启用实现不链接。

7. 借鉴的开源分层思想（clean-room 独立实现）

平台化(VESC) · 对象组合+快慢环分离(SimpleFOC) · 模块化数据流(PX4) · HAL 抽象多平台 (ArduPilot) · 配置分层(Zephyr) · BSP 独立+组件可裁(RT-Thread/ESP-IDF) · 驱动上下半部 (NuttX)。FOC 具体技术 (定点 Q15/查表三角/弱磁/死区补偿) 为通用成熟做法, 均独立实现, 不复制任何专有代码。